

Programmeringsparadigmer 2025

Kursusgang 8: Vi genbesøger emner

Opgaver til løsning og diskussion

Hans Hüttel

28. oktober 2025

Om denne kursusgang

Denne gang vil vi genbesøge emner fra tidligere kursusgange, som enten voldte hovedbrud eller var nummererede opgaver, som vi aldrig nåede at se nærmere på, fordi vi løb tør for tid. Det store mål denne gang er at hjælpe dig med at blive bedre til at forstå og anvende emner fra kursets første halvdel, og især rekursion og datatyper.

Opgaver, som vi helt sikkert vil tale om

1. **(10 minutter)** Find Haskell-udtryk, der har følgende typer:

1. $(\text{Ord } a, \text{Num } a) \Rightarrow a \rightarrow a \rightarrow [[\text{Bool}]] \rightarrow \text{Bool}$
2. $\text{Num } a \Rightarrow (t \rightarrow a, t) \rightarrow a \rightarrow a$

2. **(20 minutter)** En tidligere videnskabs- og uddannelsesminister forsøger nu at få en universitetsgrad og er travlt optaget af at lære Haskell. Ministeren forsøger at konstruere en funktion `triples`, der tager en liste af tupler (hver tuple har præcis 3 elementer) og omdanner denne liste af tupler til en tuple af lister.

`triples [(1,2,3), (4, 5, 6), (7, 8, 9)]` skal producere `( [1,4,7], [2, 5, 8], [3, 6, 9] )`.

Ministeren skrev følgende stykke kode og en typespecifikation, men løb ind i problemer:

```
triples :: Num a => [(a,a,a)] -> ([a],[a],[a])

triples [] = ()
triples [(a,b,c)] = ([a],[b],[c])
triples (x:xs,y:ys,z:zs) = [x,y,z] : Triples [(xs,ys,zs)]
```

Ret de fejl, som funktionen `triples` har. Skriv en rekursiv definition af `triples`, der rent faktisk virker. *Brug venligst rekursion og lokale deklarationer med mønstre i din løsning, og brug tilgangen beskrevet i afsnit 6.6.*

3. **(20 minutter)**

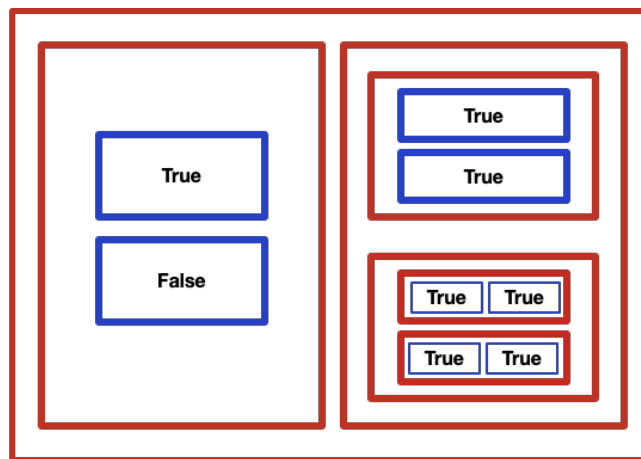
Regeringen har udarbejdet et nyt computerspil i forbindelse med kandidatreformen. Den vigtigste datastruktur er en *a-kasse*, hvor *a* er en vilkårlig type. En *a-kasse* kan være af en af følgende to typer:

- En *a-kasse* kan være *rød* og indeholde et par af *a-kasser*, eller
- En *a-kasse* kan være *blå* og indeholde en værdi.

Alle værdier, der findes i en nestet *a-kasse*, skal være af samme type.

Se Figur 1 for et eksempel.

- a) Definér en algebraisk datatype `Box`, der beskriver de gyldige former for nestede *a-kasser*. Giv et udtryk af typen `Box`, der beskriver *a-kassen* i Figur 1.
- b) En nestet *a-kasse* kaldes *ensartet*, hvis hver gang vi ser et par blå *a-kasser* et eller flere niveauer nede i *a-kassen*, så er værdierne i disse to blå *a-kasser* ens. Den nestede *a-kasse* i Figur 1 er *ikke* ensartet. Definér en funktion `ensartet`, der kan fortælle os, om en *a-kasse* er ensartet. Hvad skal typen af `ensartet` være? Er `ensartet` en polymorf funktion, og hvis den er, hvilke typer af polymorfi er der tale om?



Figur 1: En `a`-kasse fra computerspillet, der indeholder værdier af typen `Bool`.

4. (*20 minutter*) Fra lineær algebra ved vi, at et vektorrum med et indre produkt er et rum, hvor operationerne vektorsum og prikprodukt er defineret. Givet to vektorer v_1 og v_2 , er summen $v_1 + v_2$ igen en vektor, og det indre produkt $v_1 \cdot v_2$ er et tal. *Vær opmærksom på:* I lineær algebra er der meget mere til definitionen af indre produkt-rum end disse to krav, men i dette problem skal du ignorere det. Antag også, at det indre produkt er et tal af typen `Int`.

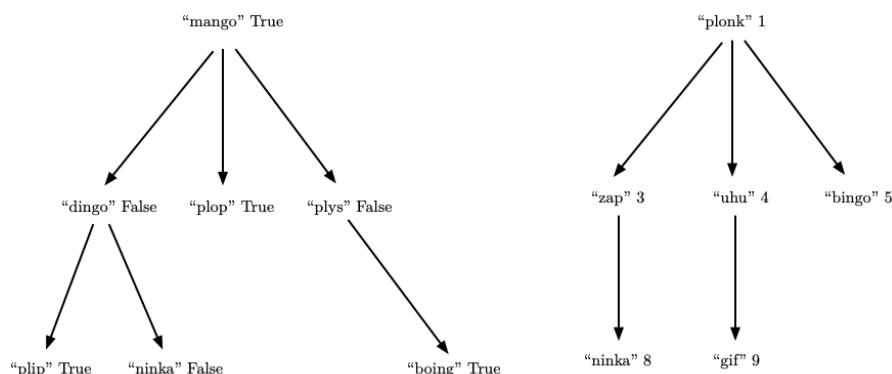
Definér en typeklasse `InVector`, hvis instanser er typer, der har de krævede operationer, og hvor vektorsum kaldes `&&&` og det indre produkt kaldes `***`. *Hint:* Hvilket afsnit i dagens tekst har du brug for her?

Find ud af, hvordan du deklarerer `Bool` som en instans af `InVector`. *Hint:* Du skal finde en definition af vektorsum og det indre produkt for sandhedsværdier.

5. (*20 minutter*) Typen `Encyclopedia` er givet ved definitionen

```
data Encyclopedia a = Entry String a [Encyclopedia a]
```

Figur 2 viser to encyklopædier.



Figur 2: To encyklopædier t_1 (venstre) og t_2 (højre)

En encyklopædi er *lagdelt*, hvis det gælder, at alle værdier på samme niveau i encyklopædien er større end værdierne på niveauerne ovenover. Som eksempel er t_2 i figur 2 lagdelt, da 8 og 9 på niveau 3 er større end værdierne 3, 4 og 5 på niveau 2 — som igen er større end værdien 1 på niveau 1.

Definér en funktion `layered`, der kan fortælle os, om en encyklopædi er lagdelt. *Hint:* Højereordensfunktionerne `all` og `map` er nyttige.

Flere opgaver, du kan løse

- a) Find en funktion eller anden værdi, der har typen

```
Fractional t1 => (t2 -> t1) -> (t2 -> t1) -> (t1 -> t3) -> t2 -> t3
```

- b) Målet med denne opgave er at definere en funktion `frequencies`, der, givet en streng s , laver en liste af par `[(x1,f1) ,... ( xk,fk)]` sådan at hvis tegnene xi forekommer i alt fi gange i listen s , så vil listen af par indeholde parret `(xi, fi)`.

Som eksempel på dette skal

`frequencies "regninger"`

give os listen

`[('r',2),('e',2),('g',2),('n',2),('i',1)]`

- a) Hvad skal funktionens type være?
 b) Brug *rekursion* til at give en definition af `frequencies`.
 c) Niveauet for en a -kasse er det maksimale antal lag af røde a -kasser, der kan omgive en blå a -kasse. Se Figur 1 for et eksempel; den nestede a -kasse, der vises her, har niveau 4. Definér en funktion `niveau`, der returnerer niveauet for en a -kasse. Hvad skal typen af `niveau` være?
 d) En sætning inden for talteori siger, at enhver reelt tal x forskelligt fra 0 kan skrives som en *kædebrøk*. Dette er et potentielt uendeligt udtryk af formen

$$x = a_0 + \frac{1}{a_1 + \frac{1}{a_2 + \frac{1}{a_3 + \frac{1}{a_4 + \frac{1}{\dots \frac{1}{a_n}}}}}} \quad (1)$$

For rationale tal vil a_i 'erne til sidst alle være 0, så kædebrøken er endelig; for irrationale tal vil kædebrøken være uendelig. Se f.eks. [1] for mere.

Skriv en Haskell-funktion `cfrac`, der, givet et reelt tal r og et naturligt tal n , finder listen af de første n tal i kædebrøken for r . Hvad skal typen af `cfrac` være?

Litteratur

- [1] Wikipedia. Kædebrøker. <https://da.wikipedia.org/wiki/Kdebrk>.